DA24B028 Aakash Aadhithya

report extra credit : favorite song

Song : **Chikiri Chikiri**

The 1-Bit Register (Bit) vs. DFF

Theoretical Comparison

In a simple DFF, after every clock tick, it captures whatever is input and outputs it until the next cycle starts. However, by using a 1-bit register, we can store the same output for as long as we want by setting load = 0 .so 1-bit register updates only when commanded meanwhile simple DFF updates after every clock tick

Do you need a Flop or a register for 1-Bit counter?

we only need a Flop because in 1-Bit counter we have to flip the bit after every clock tick that means we need preserve output till next tick and Flop exactly preserves output till next tick.

Byte-Level Access (Two 8-bit Registers) RAM2

Q1

No,If we try to build the architecture using two 8-bit registers but keep only the signals from the 16-bit register design

$$in,\ load,\ out$$

then the issue is Suppose the current value stored in the 16-bit register is

10101010 11110000

Now we want to update only the lower 8 bits to

00001111

The desired result should be

10101010 00001111

However, since there is only one `load` signal, both 8-bit registers will receive the same load signal. When `load = 1`, both registers will try to update from the input.

This may result in something like

00001111 00001111

which is incorrect, since the upper byte was not supposed to change.

To correctly update only one byte, we would need an additional control signal (such as a *byte select* signal) to choose whether the upper or lower 8-bit register should be written.

Since the original interface provides only

$$in,\ load,\ out$$

there is no way to control which 8-bit register should update.

Therefore, the architecture cannot be implemented using only the signals of the original 16-bit register design.

Q2

To fix this issue, we can introduce an additional control signal. Instead of having a single `load` signal, we use two load signals:

$$load_{high},\quad load_{low}$$

These signals control which 8-bit register should be updated.

- If `load_low = 1` and `load_high = 0`, only the lower 8-bit register is updated.
- If `load_high = 1` and `load_low = 0`, only the upper 8-bit register is updated.
- If both are `1`, then both registers are updated, which is equivalent to writing a full 16-bit value.

For example, suppose the current value stored is

$$10101010\ 11110000$$

If we want to update only the lower byte to

$$00001111$$

we set

$$load_{low} = 1, \quad load_{high} = 0$$

which updates only the lower 8 bits, giving

$$10101010\ 00001111$$

Thus, using `load_high` and `load_low` allows us to update the upper byte, lower byte, or both bytes as required.

Q3

Write Operations:

When updates are needed only for a small part of the data (for example, updating just 1 byte = 8 bits), using two 8-bit registers is useful because we can modify one register without changing the other. In contrast, if we use a single 16-bit register, updating only one byte becomes harder. We must first read the 16-bit value, change the required 8 bits, rebuild the full 16-bit value, and then write it back to the register.

On the other hand, a 16-bit register is more efficient when the data to be written is already 16 bits. In the design with two 8-bit registers, writing a 16-bit value requires two separate write operations, which takes more time. Another possible problem is that if the system fails between the two writes, the register may end up storing incomplete or inconsistent data.

Read Operations :

If the outputs of the two 8-bit registers are automatically combined, then reading a 16-bit value is similar to reading from a single 16-bit register.

However, if the design allows selecting the output of each register individually (so the output bus is 8 bits wide), then reading a single byte becomes easier. In that case, if we want the full 16-bit value, we need to perform two separate read operations and combine the bytes, which adds some delay and extra logic.

Folding into RAM8

Q1

Uses a DMux8Way to route the single load bit to the specific register addressed by the 3-bit address input. This ensures only one register is unlocked for writing. Contains eight 16-bit Registers . All eight registers receive the same 16-bit in data simultaneously, but only the one receiving the routed load signal will update its state on the clock tick Uses a Mux8Way16 to collect the outputs from all eight registers. It uses the same 3-bit address to select which register's data is sent to the final out pin.

Q2

INPUTS :

- in[16]: The 16-bit data bus carrying the value to be stored
- load: A 1-bit control signal (1 = write to the addressed register, 0 = do nothing)
- address[3]: A 3-bit number is used to select which of the 8 registers is being accessed for reading or writing

OUTPUTS :

- out[16]: A 16-bit data bus that outputs the value currently stored in the register specified by the address

Q3

To uniquely address 2^k registers you need k bits so here 3 bits are sufficient.

Memory sizing

Q1

1 Kbit = 2^{10}bits, which is 1024 bits

Q2

1 KiloByte = 2^{10}Bytes,which is 8192 bits

Q3

16 KiloBytes = 16×2^{10} Bytes which is 131072 bits

Scaling up memory

Q1

1KB = 2^{10}Bytes = 2^{13}bits
RAM8 has 2^7 bits
number of RAM8 chips = $2^{13}/2^7 = 2^6 = 64$ chips

Q2

similarly : RAM32 has 2^8 bits
Number of RAM16 chips = $2^{13}/2^8 = 2^5 = 32$ chips

Q3

similarly :RAM64 has 2^10 bits
Number of RAM64 chips = $2^{13}/2^{10} = 2^3 = 8$ chips

Address Line Management

Q1

To build 1KB of memory using RAM8, we need 64 such chips. So in total, we are using 512 16 bit registers, and to reference each of them, we need a 9 bit address because $2^9 = 512$. In the 9 bits of address, the upper 6 bits specify which one of the 64 chips, and the lower 3 bits are used to pick one of its 8 internal registers . Similarly, using RAM64,we need 8 such chips, and the 3 upper bits are to specify which one of the 8 chips, while the lower 6 bits are used to pick one of the 64 internal registers

Q2

No, the total number of address bits depends only on the size of memory; to address 512 words, you always need 9 bits.Using a larger block means you spend more address bits inside the chip and fewer bits outside.

Extra Credit : Architectural impact

Q1

Lets say if I replace RAM4K with RAM8, then i need 512 wires and In[16] data has to physcally stretch across a huge area of chip to touch very single one those chips so it take more power and more time to flip the wire from 0 to 1.But in RAM4K all registers are tightly packed and internal wires are microscopic.there fore by using larger blocks of memory reduces the total length of wires globally.AI chips need to move terabytes of data per second so if the memory was built using smaller chips the heat generated by load wires would melt the chip and by using large blocks of memory heat produced is almost minimal.

Q2

Using 1K RAM8 chips is costlier to design and implement.

RAM8 is a very small memory block, suppose to build a 16KB memory we have to combine many RAM8 chips step by step to form larger memories (like RAM64, RAM512, RAM4K, etc.) before finally reaching 16KB. At every stage we need extra selection logic to choose which chip is active. This increases the amount of hardware and makes the circuit more complex.

So if i already have larger blocks like RAM4K then i just need to combine two of such chips to get 16KB and you dont have to invest time and money in building RAM4K which is necessary when RAM8 is used as basic block of memory

Documentation

Design Choice and Rationale for Wallace Tree Multiplier

Multiplying two 16-bit numbers a and b can be viewed as the addition of several shifted partial products. Each bit of one operand is multiplied with every bit of the other operand, producing intermediate rows of bits.

$$a \times b = \sum_{i=0}^{15} (a << i) \quad \text{if } b_i = 1$$

Since multiplying two 16-bit numbers can produce a result up to 32 bits, the output is stored as two 16-bit words: one representing the upper 16 bits and the other representing the lower 16 bits.

In the worst case there can be up to 16 partial products. A straightforward implementation would add these partial products sequentially, which would require many addition stages(number of addition stages can be atmost 16). Instead, we use a Wallace Tree structure which reduces multiple partial products in parallel.

Wallace Tree Reduction

In the Wallace tree, the partial product rows are grouped and reduced using FullAdders and HalfAdders. A FullAdder takes three input bits and produces a sum and a carry, while a HalfAdder takes two input bits and produces a sum and carry.

- The **sum** remains in the same column
- The **carry** moves to the next column

$$3 \text{ rows } \rightarrow 2 \text{ rows}$$

By repeatedly applying these reductions, the number of rows decreases after each stage until only two rows of bits remain.

Final Addition Stage

After the Wallace tree reduction, the remaining two rows are added using a **ripple carry adder** implemented using FullAdders. The carry propagates from the least significant bit to the most significant bit to produce the final result.

The final 32-bit product is divided into two outputs:

- **lo** : lower 16 bits
- **hi** : upper 16 bits

This structure allows many partial products to be reduced in parallel, which significantly reduces the number of addition stages compared to sequential addition.

Lets analyze time taken to reach final stage:

- every stage takes constant time $\mathcal{O}(1)$ because we are adding them parellally
- after every stage n, the rows become approximately $\frac{2n}{3}$.
- lets say there are total k stages ,then

$$(\frac{2}{3})^k.n = 1 \tag{1}$$

$$k = \log_{1.5} n \tag{2}$$

- Wallace tree reduced time complexity from $\mathcal{O}(n)$ to $\mathcal{O}(\log n)$ and in our case $n = 16$ which takes only one cycle to compute so $\mathcal{O}(1)$

Instruction Encoding for Multiplication

In the Hack ALU, the operation performed is determined by six control bits:

$$(zx, nx, zy, ny, f, no)$$

These bits control how the ALU processes its inputs and decide which operation is performed.. In the original design, these bits support 18 arithmetic and logical operations, such as addition, subtraction, bitwise AND, and bitwise OR.

To extend the ALU with a hardware multiplication operation, we selected an unused combination of the six control bits

Our multiplier produces a 32-bit result, while the ALU can output only a 16-bit value at a time. Therefore, two instructions are used: one to return the lower 16 bits and another to return the upper 16 bits
for lower 16 bits

$$(zx, nx, zy, ny, f, no) = (0, 1, 0, 1, 1, 0)$$

for upper 16 bits

$$(zx, nx, zy, ny, f, no) = (0, 1, 0, 1, 1, 1)$$

Thus, obtaining the complete product requires executing both instructions sequentially This control pattern does not belong to any of the existing ALU operations,making it suitable for introducing a new instruction.

when these combinations are selected, ALU computes the product of the numbers, and clearly, it does not overlap with any other instruction, so adding this multiplier does not affect the other 18 operations.

If one multiplicand is of the format 2^N or $2^N + 1$

In general, the number of partial products in a multiplier depends on the number of set bits in the multiplicand.

If one multiplicand is exactly of the form 2^N, then it has only one set bit. Therefore, the multiplication

$$a \times 2^N$$

is equivalent to shifting a left by N bits. In this case, we do not need a full multiplier

such as a Wallace tree instead a simple shift operation is sufficient.

If the multiplicand is of the form $2^N + 1$, then it has two set bits. The product becomes

$$a(2^N + 1) = (a \ll N) + a$$

So the multiplication can be implemented using one shift operation and one addition. Therefore, the multiplier architecture can be simplified since a full Wallace tree multiplier is unnecessary.